Chapter 4: Linux Kernel

Architecture • Build • System Calls • Embedded Context

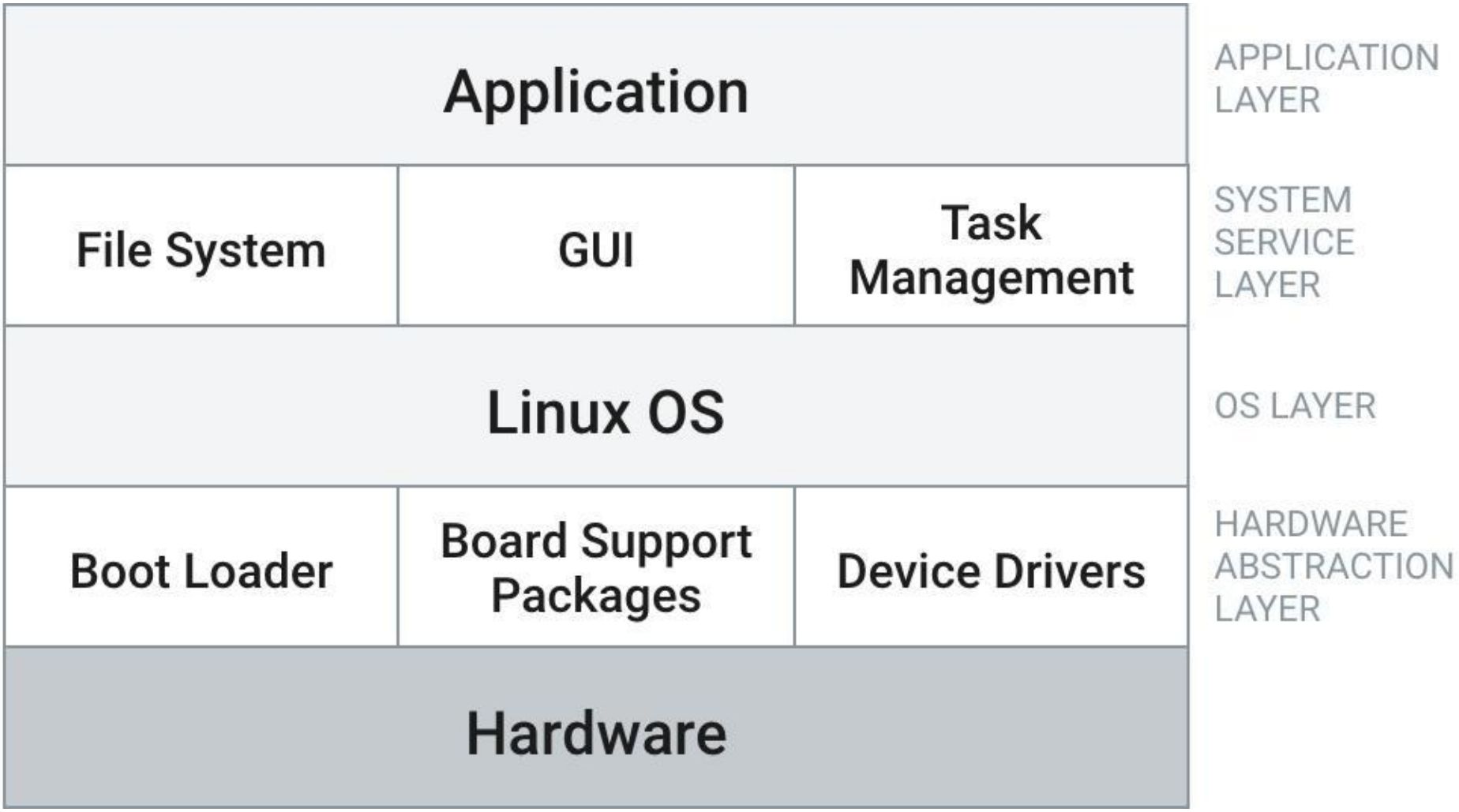
Learning Objectives

- Understand Linux kernel architecture deeply
- Explain system call flow
- Analyze kernel build system
- Apply kernel knowledge in embedded systems

Role of the Kernel

- Interface between hardware and user applications
- Manages CPU, memory, devices
- Ensures process isolation
- Provides system services via system calls

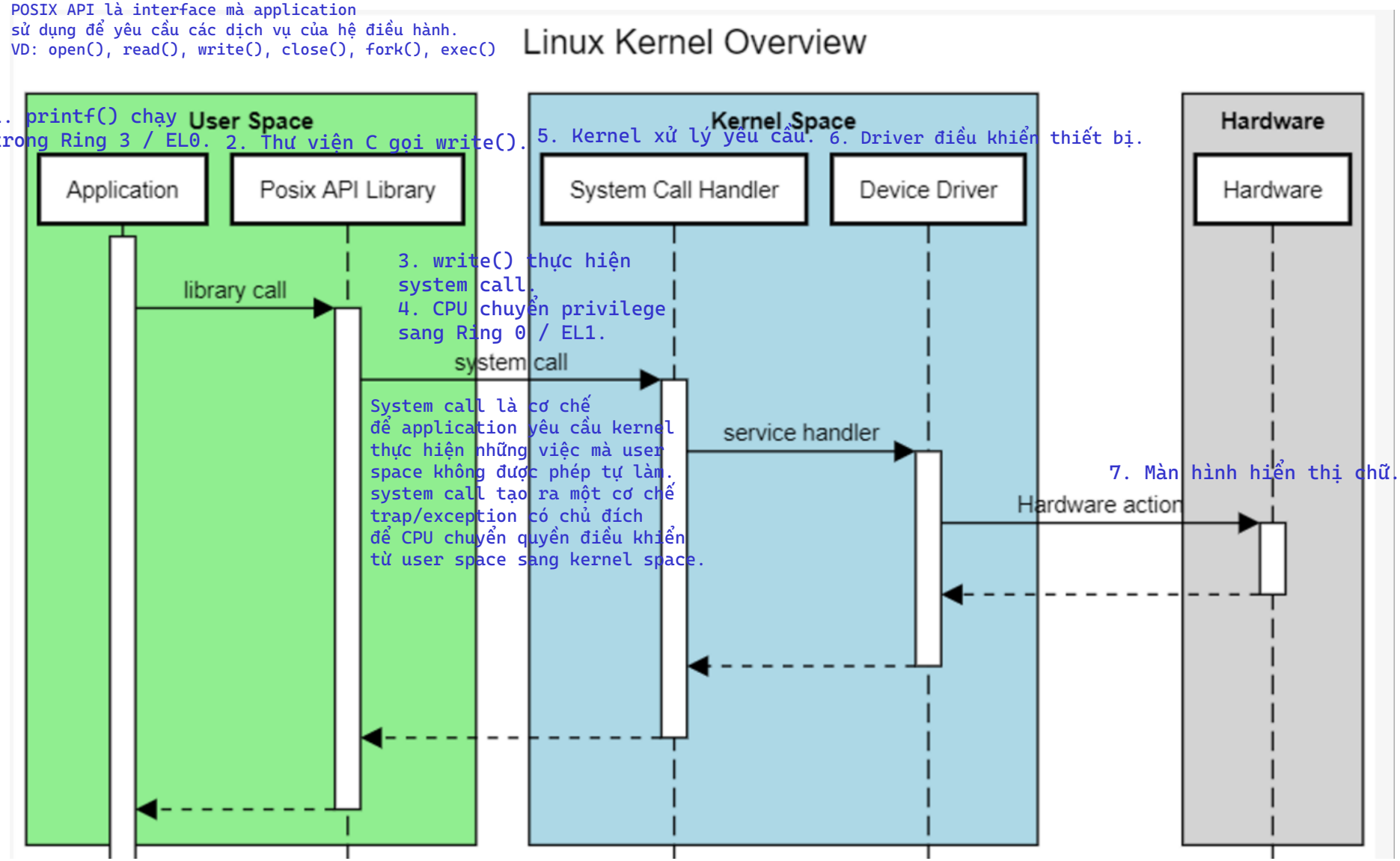
Linux Architecture Overview



Kernel là lớp trung gian đặc quyền giữa Applications và Hardware, cung cấp các dịch vụ hệ thống thông qua System Calls và xử lý các sự kiện từ Hardware thông qua Interrupts, Traps và Exceptions.

System Call Flow

POSIX API là interface mà application sử dụng để yêu cầu các dịch vụ của hệ điều hành.
VD: open(), read(), write(), close(), fork(), exec()



System call là cơ chế để application yêu cầu kernel thực hiện những việc mà user space không được phép tự làm. system call tạo ra một cơ chế trap/exception có chủ đích để CPU chuyển quyền điều khiển từ user space sang kernel space.

CPU cung cấp hardware-enforced privilege để ngăn Application truy cập trực tiếp tài nguyên hệ thống; Kernel chạy ở privilege level

Privilege Levels

Giả sử một chương trình người dùng có thể:
ghi trực tiếp vào RAM của hệ điều hành, tắt ngắt (interrupt)
truy cập ổ đĩa thô ... thì chỉ cần một lỗi nhỏ là toàn bộ hệ thống sập.
Vì vậy CPU cung cấp các mức đặc quyền (privilege levels).

x86

Ring càng nhỏ thì quyền càng cao.

↳ uses "rings"

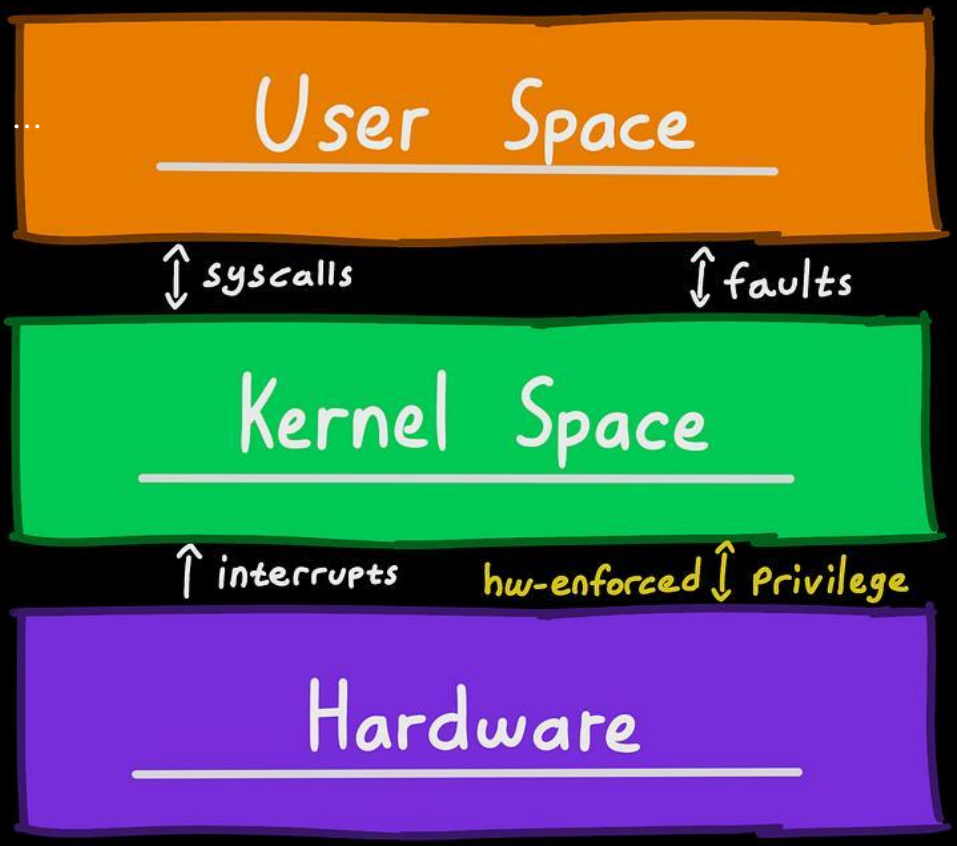
user apps: chrome, python, ...
CPU chặn xử lý interupt

↳ ring 3 is least privileged

↳ ring 0 is most privileged
kernel

↳ ring 1 & 2 exist but unused

ring 0: truy cập toàn bộ RAM
cấu hình MMU
điều khiển thiết bị
xử lý interrupt



arm

↳ uses "ELs" Exception Levels

↳ EL0 is least privileged

↳ EL1 is kernel more privileged

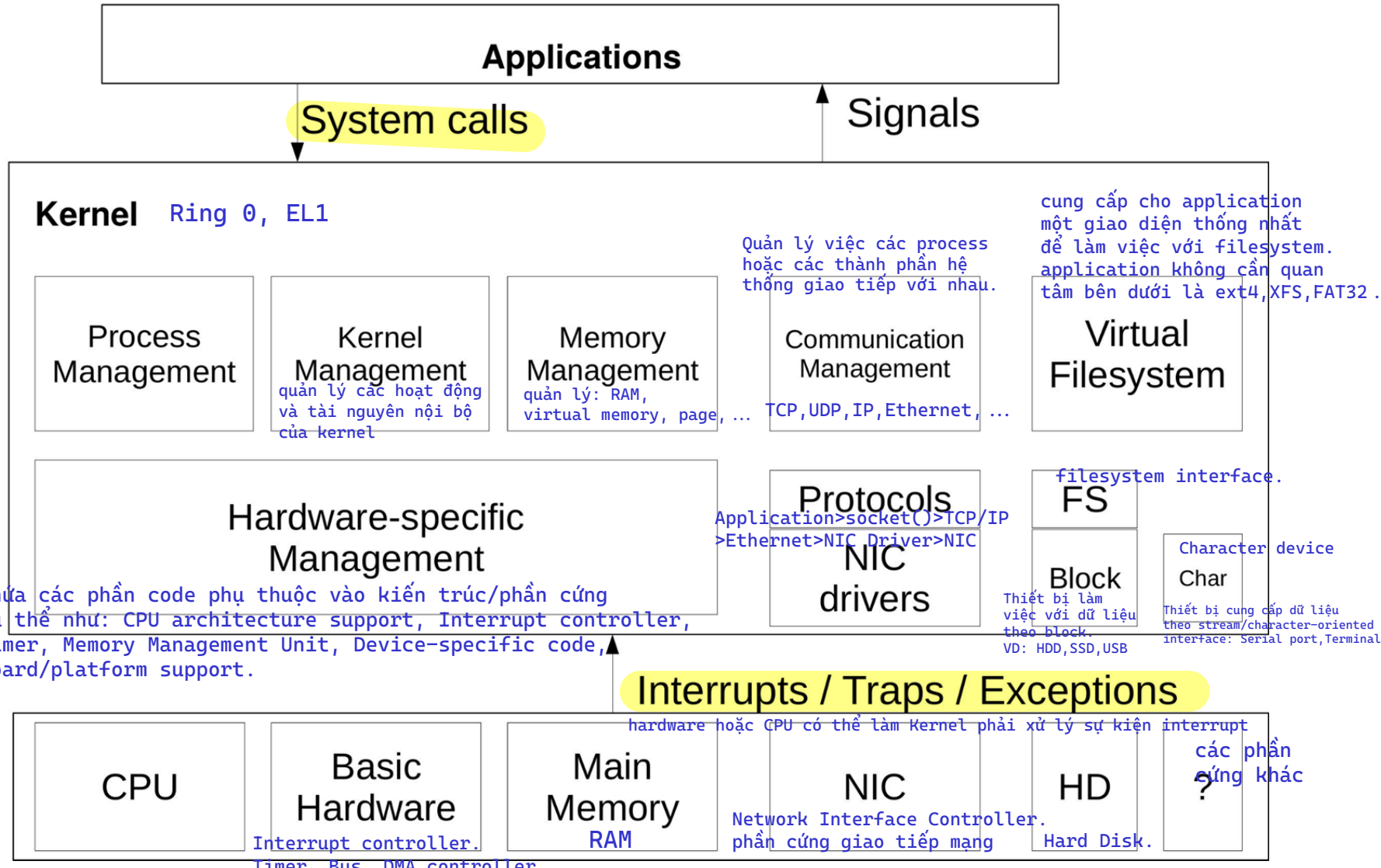
↳ EL2 is even more privileged
EL2 dùng cho ảo hóa máy chủ

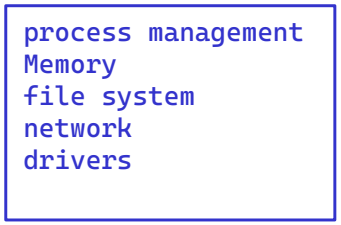
↳ EL3 is most privileged
EL3 dùng cho bảo mật cấp phần cứng

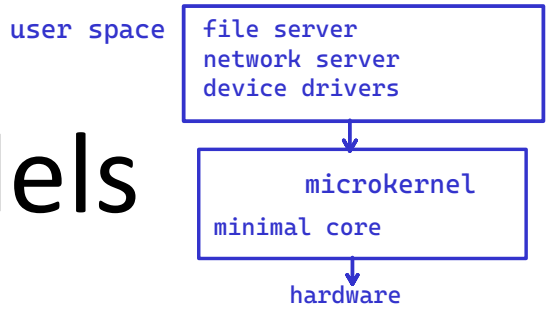
CPU dùng hardware-enforced privilege để bảo vệ hệ thống,
còn kernel là phần mềm duy nhất thường chạy ở mức đặc quyền cao
và cung cấp dịch vụ cho mọi ứng dụng thông qua system call.

*Trap thường là một sự kiện được tạo ra có chủ đích bởi instruction
*Exception thường xảy ra khi CPU phát hiện một điều kiện đặc biệt trong quá trình thực thi instruction.

Kernel Components







Kernel Design Models

các thành phần chính của Kernel được tích hợp và chạy trong kernel space, có thể gọi trực tiếp lẫn nhau.

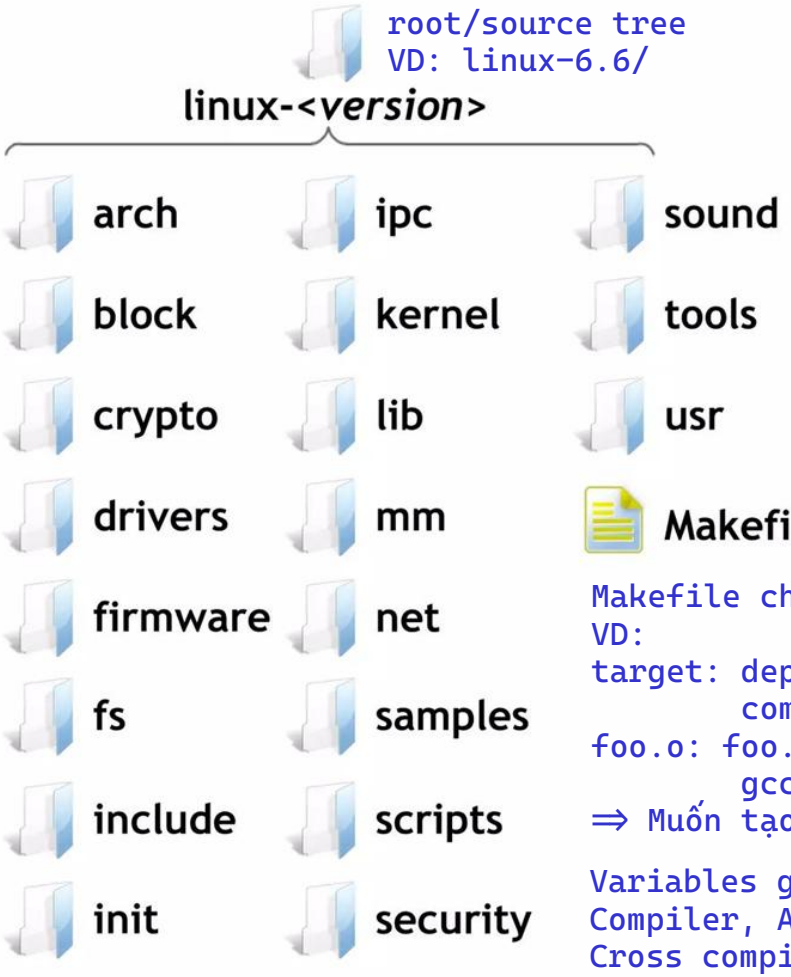
- Monolithic: Linux (fast, integrated)
- Microkernel: minimal core
- Linux advantages: performance, flexibility
- Trade-off: complexity

→ Modern Linux is no longer a purely monolithic kernel.

It combines monolithic performance with modularity, sandboxing, and strong isolation mechanisms.

Linux Kernel: core, network module, usb module, storage module, device driver module, ...
các module có thể được tích hợp vào kernel khi cần.

Kernel Source Structure



- This is top level Makefile for the whole source tree
- Contains useful rules and variables like default gcc compilation flags

là Makefile cấp cao nhất,
điều khiển quá trình build của toàn bộ Linux Kernel.

Makefile chứa rất nhiều rule.
VD:

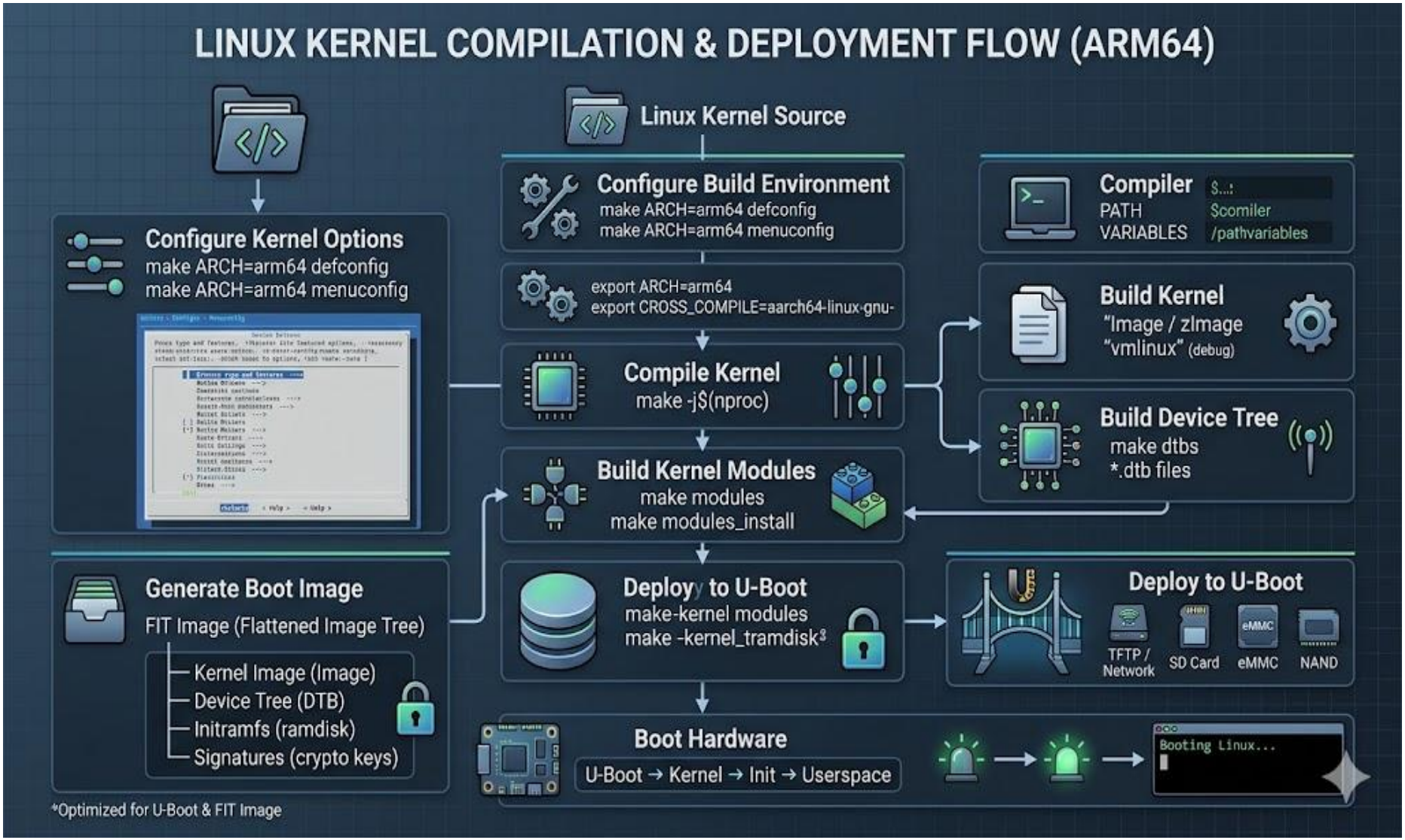
```
target: dependencies
        command
foo.o: foo.c
        gcc  ...  -c foo.c -o foo.o
```

⇒ Muốn tạo foo.o thì cần foo.c, và phải compile foo.c

Variables giúp kernel build system biết:
Compiler, Assembler,Linker,Compiler flags,Architecture
Cross compiler,Output directory, ...

Thư mục	Chức năng chính
`arch/`	Code phụ thuộc kiến trúc CPU
`block/`	Block I/O subsystem
`crypto/`	Cryptographic API/implementation
`drivers/`	Device drivers
`firmware/`	Firmware liên quan đến thiết bị
`fs/`	Filesystem
`include/`	Header files
`init/`	Kernel initialization
`ipc/`	Inter-process communication
`kernel/`	Core kernel
`lib/`	Các thư viện/hàm tiện ích dùng trong kernel
`mm/`	Memory management
`net/`	Networking
`samples/`	Ví dụ sử dụng kernel APIs
`scripts/`	Script và công cụ hỗ trợ build
`security/`	Kernel security framework
`sound/`	Audio subsystem
`tools/`	Các công cụ hỗ trợ phát triển/kernel
`usr/`	Một số thành phần liên quan userspace/initramfs

Kernel Build Flow



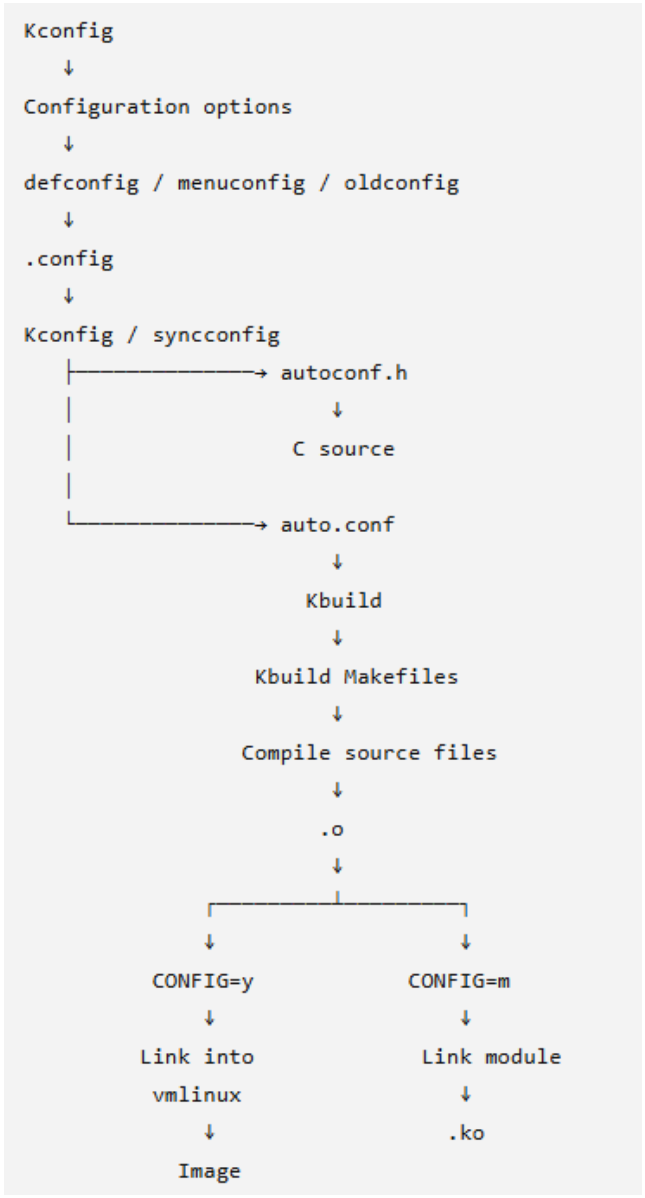
```
Linux Source
    ↓
Configuration
    ↓
Compile
    ↓
Build DTB
    ↓
Build Modules
    ↓
Create Boot Image
    ↓
Copy to board
    ↓
U-Boot
    ↓
Kernel
    ↓
Init
    ↓
Userspace
```

```
Bước 1: Linux Source, VD: linux-6.6/
Bước 2: Chọn cấu hình
make ARCH=arm64 defconfig
tạo .config
VD: CONFIG_USB=y
CONFIG_EXT4=y
CONFIG_I2C=y
CONFIG_NET=y
Bước 3: Thiết lập Compiler
VD: export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-
Dùng cross compiler vì máy build: x86 PC
target: ARM64 Board
Bước 4: Compile
make -j$(nproc)
Kernel build system: .c .s → .o → .a → vmlinux
Bước 5: Tạo file boot
ARM64: make Image tạo: arch/arm64/boot/Image
hoặc make Image.gz tạo Image.gz
Bước 6: Build Device Tree
make dtbs tạo: arch/arm64/boot/dts/
VD: bcm2711-rpi-4-b.dtb
zynqmp.dtb
stm32mp157.dtb
Bước 7: Build Modules
```

```
*Device Tree mô tả phần cứng:
CPU,RAM,GPIO,UART,SPI,I2C,Ethernet
VD:
uart0 {
    status = "okay";
};
Kernel đọc DTB để biết board có các thiết bị gì
```

Kernel Build Flow Command

1. **Kconfig** files define the options.
2. User selects options, creating **`.config`**.
3. `make` converts `.config` into **`autoconf.h`** (for C code) and uses it in **Makefiles** (for compilation).
4. The compiler produces either a **built-in** feature or a **`.ko` module**.



1. Kconfig trong source tree định nghĩa những tùy chọn cấu hình của Kernel, chẳng hạn như hỗ trợ USB, networking, filesystem, device driver ...
Mỗi tùy chọn được biểu diễn bởi một configuration symbol có dạng CONFIG_*.
lựa chọn cấu hình Kernel thông qua các lệnh:
make defconfig
make menuconfig
make oldconfig

2. Sau khi lựa chọn, hệ thống sẽ tạo ra file:
.config

3. .config chứa cấu hình cuối cùng của Kernel. Mỗi configuration symbol thường có một trong ba trạng thái: y → built-in, m → module, n → disabled
VD:
CONFIG_USB=y, CONFIG_USB_STORAGE=m, CONFIG_DEBUG_FS=n
⇒ USB được tích hợp trực tiếp vào Kernel, USB Storage được build thành module và Debug FS không được build.

4. Sau khi .config được tạo, hệ thống Kconfig/syncconfig tạo ra các file cấu hình sinh tự động để phục vụ quá trình build. Trong đó có: include/generated/autoconf.h
file này cung cấp các macro CONFIG_* cho mã nguồn C của Kernel.

5. Kbuild kết hợp .config với các Kbuild Makefile trong source tree để quyết định những source file nào cần được build và chúng sẽ được build theo dạng nào.
*Kconfig chịu trách nhiệm mô tả và xử lý cấu hình, còn Kbuild chịu trách nhiệm tổ chức quá trình build.
VD Kbuild Makefile:
obj-$(CONFIG_MY_DRIVER) += my_driver.o
nếu CONFIG_MY_DRIVER=y ⇒ my_driver.o được đưa vào Kernel built-in.
nếu CONFIG_MY_DRIVER=m ⇒ driver được build thành my_driver.ko
nếu CONFIG_MY_DRIVER=n ⇒ driver không được build

* Đối với một thành phần được cấu hình: CONFIG_X=y
các object file của thành phần đó được liên kết vào Kernel, cuối cùng tạo nên: vmlinux, từ đó có thể tạo Kernel image phù hợp với architecture, chẳng hạn đối với ARM64: arch/arm64/boot/Image

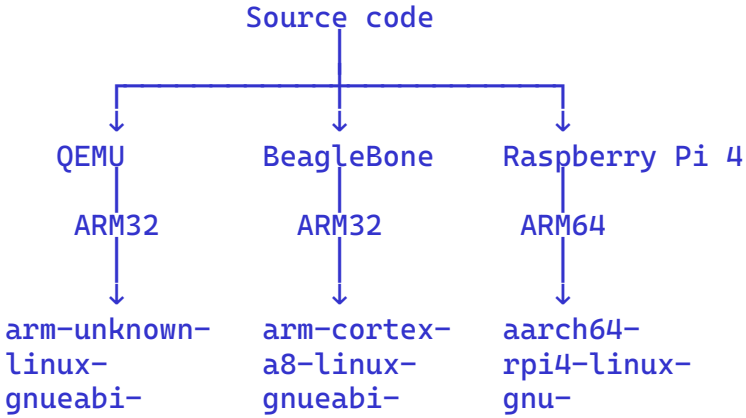
* Đối với thành phần được cấu hình: CONFIG_X=m
các object file được liên kết thành một Kernel module: X.ko
Module này có thể được cài vào root filesystem, thường dưới: /lib/modules/<kernel-version>/ và được nạp vào Kernel khi hệ thống đang chạy bằng các công cụ như: modprobe X

Cross Compilation

Platform	Target Architecture	Toolchain Prefix	Key Use Case	Tool Used
công cụ emulation/ virtualization		target: ARM + Linux + EABI	Generic ARM ⇒ code có thể chạy trên nhiều ARM CPU	
QEMU	ARMv5TE (ARM926EJ-S)	32 bit arm-unknown-linux-gnueabi-	Virtualized ARM testing	QEMU + crosstool-NG
dùng arm-unknown-linux-gnueabi-gcc hello.c -o hello thay vì gcc hello.c -o hello				
		compiler tạo code tối ưu cho Cortex-A8		
BeagleBone Black	ARM Cortex-A8 (ARMv7-A)	arm-cortex_a8-linux-gnueabi-	Industrial prototyping	crosstool-NG
arm-cortex_a8-linux-gnueabi-gcc hello.c -o hello				
		ARM 64-bit		
Raspberry Pi 4	ARM Cortex-A72 (ARMv8-A, 64-bit)	aarch64-rpi4-linux-gnu-	64-bit embedded development	crosstool-NG
aarch64-rpi4-linux-gnu-gcc hello.c -o hello				

arm-unknown-linux-gnueabi-
arm: CPU architecture (ARM 32 bit)
unknown: Vendor/platform vendor không được xác định cụ thể.
linux: target OS
gnueabi: GNU + EABI, EABI = Embedded Application Binary Interface
gnueabi ⇒ ABI cho Linux ARM 32-bit theo GNU EABI.

crosstool-NG: công cụ giúp tạo cross-toolchain
VD: muốn có ARM32 + Linux + EABI
cấu hình crosstool-ng:
Architecture: ARM
OS: Linux
ABI: EABI
sẽ tạo ra:
arm-unknown-linux-gnueabi-gcc - compiler (GNU Compiler Collection)
arm-unknown-linux-gnueabi-ld - linker (GNU Linker)
arm-unknown-linux-gnueabi-as - assembler (GNU Assembler)
...
mối liên hệ:
1. Mã nguồn (.c) → Gọi gcc → Tạo Mã hợp ngữ (.s)
2. Mã hợp ngữ (.s) → Gọi as → Tạo File đối tượng (.o)
3. File đối tượng (.o) → Gọi ld → Tạo File thực thi



Cùng một source code nhưng compiler tạo ra binary khác nhau.

aarch64-rpi4-linux-gnu-
aarch64: ARM 64-bit
rpi4: target/platform/vendor identifier
linux: Linux OS
gnu: GNU ABI/toolchain environment

arm-linux-gnueabi- :thường được dùng cho ARM 32-bit Linux.
aarch64-linux-gnu- : toolchain phổ biến cho ARM 64-bit Linux.

Toolchain	Kiến trúc
`arm-linux-gnueabi-`	ARM 32-bit
`arm-linux-gnueabihf-`	ARM 32-bit + hard float
`aarch64-linux-gnu-`	ARM 64-bit

Kernel in Embedded Systems

cấu hình Linux Kernel chỉ chứa những thành phần cần thiết cho thiết bị.

- Customized kernel
  - Disable unused drivers, filesystems, and debugging features to reduce flash and memory usage.
    - CONFIG_CC_OPTIMIZE_FOR_SIZE=y
- Remove unused features
- Optimize size and boot time
  - PREEMPT_RT improves determinism and reduces interrupt/scheduling latency.
    - CONFIG_PREEMPT_RT=y
  - Removing unnecessary initialization steps reduces system startup latency.
- Hardware-specific configuration
  - Device Tree
  - GPIO
  - SPI/I2C/UART
  - interrupt configuration

VD k cần NTFS

chỉ cần thiết trong quá trình dev

compiler optimization nhằm giảm kích thước code.

Thời gian phản hồi có thể dự đoán được.

PREEMPT_RT cải thiện đáng kể khả năng đáp ứng thời gian thực và tính xác định của Linux, nhưng yêu cầu hard real-time tuyệt đối vẫn cần xem xét toàn bộ hệ thống và worst-case behavior.

Tắt nhưng thứ không cần trong Kernel configuration:
make menuconfig
⇒ giảm kích thước kernel, giảm RAM, giảm thời gian khởi động

Kernel Debugging (Intro)

ghi lại thông tin mà kernel muốn nói cho chúng ta biết.

VD: printk(KERN_INFO "My driver started\n");
Kernel sẽ ghi thông tin này vào kernel log buffer. Sau đó ở user-space có thể xem bằng: dmesg

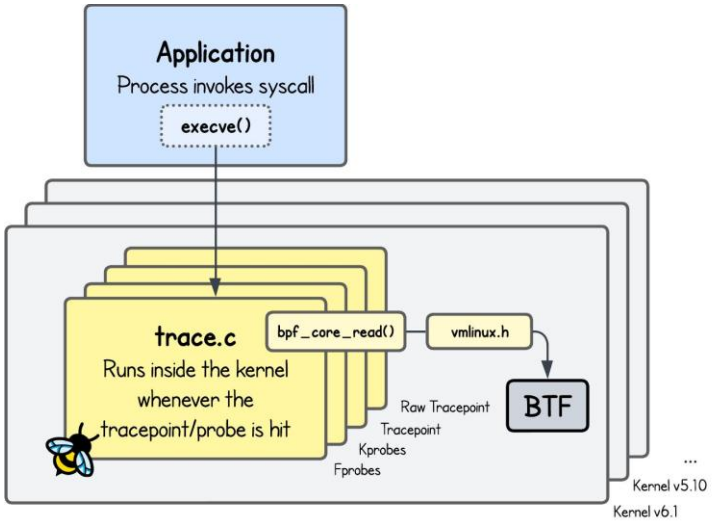
- Logging: printk(), dmesg
- Tracing: ftrace, tracepoints, function_graph tracer

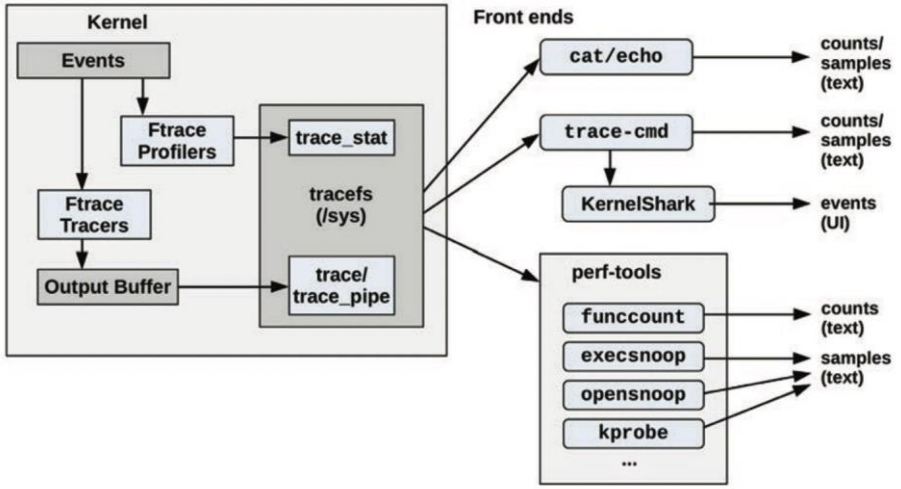
Kernel thực sự gọi những hàm nào và theo thứ tự nào?

CPU đang tốn thời gian/tài nguyên ở đâu?

Performance Monitoring Unit

- Profiling: perf, hardware PMU counters
- Modern Observability: eBPF, bpftrace, BCC





Summary

- **Kernel Organization:** Understanding the structural breakdown of the kernel components is crucial for navigating and customizing this massive project.
- **Build System:** Complexity is managed smoothly through a combination of intuitive graphical configuration editors and recursive makefiles.
- **Holistic View:** The Linux kernel is only one piece; a functional embedded design requires many complementary software components